

Prototype Design Pattern — Interview & Engineering Handbook

PHP 8.3 · Laravel · Node.js · Saudi Arabia | Dubai/UAE | Malaysia | India
Tier-2 | India Tier-1/60LPA+

Interview Prep Reference · Updated July 2026

Prototype Design Pattern

A note on length before you start: Prototype is a **Low-frequency** pattern per the research in `design-patterns-frequency-guide-expanded.md` — it is not a headline LLD round anywhere in your five target markets. This handbook is deliberately shorter than what you'd want for Strategy, Factory, Observer, or Singleton (your Very High priority patterns). Don't mistake the brevity for incompleteness — it's calibrated to how much interview time this pattern actually earns. Spend your remaining prep budget on the Very High patterns.

Companion file: all runnable code referenced here lives in `Prototype.php`, same folder. This document is theory and rehearsal; the code file is the lab.

⚡ **FAST TRACK — read this every time you revisit this pattern**

Part 1 — 60-Second Recall Card

Category	Creational (GoF)
One-liner	Create new objects by cloning an existing, fully-configured instance instead of building from scratch

Core mechanism (PHP)	<code>clone</code> keyword + <code>__clone()</code> magic method
Problem it solves	Repeated expensive object construction (I/O, parsing, crypto)
Trigger phrase	"Expensive setup," "same object thousands of times," "template + small variation"
Anti-trigger	Cheap objects; genuinely varying types (→ Factory); session/auth/payment-transaction state (never clone this)
#1 interview follow-up	Shallow vs. deep copy — PHP's <code>clone</code> is shallow by default; nested objects are shared references until <code>__clone()</code> says otherwise
Closest confused patterns	Factory (creates fresh, doesn't need an existing instance), Builder (step-by-step, no existing instance), Singleton (exactly one instance, ever — actively blocks cloning)
Memory hook	<i>Prototype = rubber stamp.</i> Carving the stamp (construction) happens once; stamping paper (cloning) is cheap and repeated.

Part 2 — Market Calibration

Pulled directly from `design-patterns-frequency-guide-expanded.md` — no claim here that isn't traceable to that research.

Market	Confirmed as a named/standalone topic?	Realistic role
Saudi Arabia	No	Not observed in any researched company's LLD round. If it appears, expect it buried inside a caching or template-duplication discussion (e.g. HungerStation's "architecture and design patterns" round), not as its own prompt.
Dubai/UAE	No	Same — not a named topic at Careem, Property Finder, Talabat, or any other researched UAE company. Possible follow-up inside a caching-layer or templating discussion.
Malaysia	No	Not observed. Malaysia's confirmed emphasis is Singleton (live-coded) and Strategy — Prototype has no footprint in the research.
India Tier-2	No	Not observed as a standalone topic across Razorpay, PhonePe, Swiggy, CRED, Freshworks, or the other 24 researched companies. The closest adjacent confirmed topics are LRU Cache design (PolicyBazaar, CRED) and multilevel caching (PhonePe) — genuinely related to "avoid rebuilding expensive state," but tested as caching, not as Prototype by name.
India Tier-1/60LPA+	No	Not observed across Amazon, Google, Microsoft, Atlassian, Rippling, or the other 22 researched companies either.

Honest bottom line: across ~145 companies researched in five markets, Prototype was not confirmed as a named interview topic anywhere. That doesn't mean skip it — it means: (1)

know it well enough to recognize and name it *if* a caching/templating/object-duplication LLD problem naturally calls for it, (2) know the shallow-vs-deep-copy mechanics cold, because that specific gotcha is genuinely general-purpose PHP/OOP knowledge that can surface in *any* round involving object copying, regardless of whether the interviewer calls it "Prototype," and (3) don't over-invest prep time here relative to your Very High priority patterns.

Part 3 — Recognition, Decision Tree & When NOT to Use

Requirement phrases that signal Prototype: - "Object setup/initialization is expensive or slow" - "We create the same kind of object thousands of times per batch/request" - "Most fields are identical; only two or three vary per instance" - "We already have a fully-configured object — we just need another one like it"

Decision tree:

```
Is construction meaningfully expensive (I/O, crypto, parsing, computation)?
├─ No → just use `new`. Stop.
└─ Yes → Are you creating MANY structurally-similar instances from shared baseline state?
    └─ No (types genuinely differ) → Factory problem, not Prototype. Stop.
    └─ Yes → Does the object graph contain mutable nested objects (not just scalars)?
        └─ No → default shallow clone is fine. Done.
        └─ Yes → clone WITH a correctly-cascading __clone() override for every mutable nested object. Done.
```

When NOT to use it (say these unprompted — it's a strong signal): - The object is cheap to construct — cloning adds overhead and complexity for nothing. - Object types genuinely vary per request (PaymentMethod: CreditCard vs. PayPal vs. BankTransfer) — that's Factory's job. - **The object holds session, auth, or payment-transaction state.** Cloning risks silently duplicating a reference to sensitive shared state across independent operations — a correctness *and* security concern, not a style preference.

✓ **Before you move on:** (1) Name one requirement phrase that should make you think "Prototype." (2) Name one type of object you should never clone, and why.

Part 4 — Cheat Sheet & Multi-Length Pitch

PHP mechanism	<code>clone</code> + optional <code>__clone()</code>
Default copy behavior	Shallow — scalars by value, objects by shared reference
<code>clone</code> calls <code>__construct()</code> ?	No — never
<code>__clone()</code> runs when?	After the shallow copy already exists, on the new object — a repair hook, not an interceptor
Cooperates with	Factory (often builds the initial prototype), Registry (keys multiple variants)

Structural fix beyond <code>__clone()</code> discipline	Make nested state immutable by default — removes the need for deep-copy logic entirely
--	--

30 seconds: "Prototype creates new objects by cloning an existing, fully-configured instance instead of constructing from scratch. It's used when construction is expensive — pay the setup cost once, clone cheaply after. The catch: PHP's `clone` is shallow by default, so nested objects need an explicit deep-copy in `__clone()`, or you get shared-state bugs."

1 minute: The 30-second version, plus: "A concrete example is invoice generation — branding, tax rules, and templates are expensive to assemble and identical across thousands of orders, so you build one prototype per tax jurisdiction, clone it per order, and overwrite only order-specific fields. The most common bug is forgetting a nested object like a tax profile is shared by reference after a shallow clone — I always pair this with a test that asserts clone independence using identity checks, not value equality."

3 minutes: The 1-minute version, plus: PHP object internals in one sentence (objects are heap-allocated, reference-counted handles; `clone` allocates a new object and bitwise-copies property values — copy-on-write applies to arrays/strings, not to the object graph itself); the registry variant for multiple variants (per-country tax config) with event-driven invalidation; the explicit anti-pattern of cloning security/transaction-sensitive state; the concurrency nuance between PHP-FPM's per-request isolation and long-running processes like Swoole, where an incomplete `__clone()` becomes a genuine race condition.

10 minutes: The 3-minute version, plus: the full refactoring journey from a naive constructor to a tested, registry-backed production version (Part 19); the SOLID analysis; the comparison table against Factory/Builder/Singleton/Memento; the argument that immutability is the highest-leverage fix for the entire shallow-copy bug class, higher-leverage than `__clone()` discipline alone; and the honest fact (Part 2) that this pattern doesn't headline any interview round in your target markets — so the 10-minute version is mostly useful for showing depth if it comes up as a follow-up, not for a dedicated round.

Part 5 — Timed Mock Drill

Prompt (give yourself 45 minutes, no notes): *"We run a multi-region invoice service. Every invoice requires company branding, a tax profile (VAT/GST rate), a currency, and a PDF template — all identical for every order from the same country, loaded from a config service, a tax-rules service, and a template store respectively. During a flash sale we process 50,000 orders in a short window and invoice generation is now the dominant contributor to processing latency. Design a solution."*

Time-boxed sub-steps: - **0-5 min:** Clarify — is the bottleneck construction cost or something else (DB writes, PDF rendering itself)? State your assumption explicitly if the interviewer doesn't answer. - **5-15 min:** Identify the expensive/cheap split (shared config vs. order-specific fields). Sketch the class shape and the registry concept out loud before coding. - **15-30 min:** Write the `InvoicePrototype` class with a correct `__clone()`, and a keyed registry that never returns the stored master directly. - **30-40 min:** Handle the follow-up twist: *"Tax rules just changed for one country mid-sale. How does this update without redeploying?"* — this is where most candidates either freeze or bolt on an ungoverned mutation. A strong answer proposes

event-driven registry invalidation. - **40-45 min:** State the trade-off you're making out loud, unprompted: registry staleness is now a monitored concern that didn't exist before.

Self-grading rubric — a bar-raiser-caliber interviewer is scoring: - Did you separate expensive/shared fields from cheap/instance-specific fields *before* writing code, or discover it while coding? - Did you get the registry to return a clone, never the master, without being prompted? - Did you preemptively mention the shallow-copy risk for the nested tax-profile object, or only after a hint? - Did you handle the "tax rules changed mid-sale" twist with a structural answer (event-driven invalidation) rather than a hand-wave ("we'd just update it")? - Did you name the trade-off (registry staleness as a new operational concern) unprompted?

✓ **Before you move on:** (1) What's the one design decision in this drill most candidates get right? (2) What's the one follow-up most candidates fumble?

Part 6 — Pattern Recognition Drill

Scenario count here is intentionally short — Prototype's real footprint in your target markets is thin (Part 2), so a long drill would itself be padding. Five scenarios, covering the genuine confusion points with Factory, Builder, and Singleton.

Scenario 1: "Design a notification system that sends templated emails/SMS in five languages. Branding and localized copy are loaded once; recipient and event data vary per send." → **Prototype** (clone a per-locale template, overwrite recipient fields). *Not Factory* — the concrete "type" of notification object doesn't vary, only its data; there's no branching decision to encapsulate. *Not Builder* — there's no multi-step optional construction, just cheap duplication of an already-complete object.

Scenario 2: "Design a payment processing system supporting credit card, PayPal, and bank transfer, chosen at runtime based on the customer's selection." → **Factory**, not Prototype. The concrete type genuinely varies per request — this is a creation-logic decision, not a duplication-of-similar-objects problem. Prototype would be the wrong pitch here even though "creating objects" is involved.

Scenario 3: "Design a report-generation service. Reports share a chart style and company logo; each report has a different date range and metric set." → **Prototype**. Same shape as invoice generation — expensive shared styling, cheap varying payload. *Not Builder* — a Builder would fit if report construction had many optional, combinable steps; here it's closer to "clone the styled template, set two fields."

Scenario 4: "Design a global app-config object that must exist exactly once for the lifetime of the application, and be accessible from anywhere." → **Singleton**, not Prototype — and worth stating explicitly that Singleton typically makes `__clone()` **private** specifically to *block* what Prototype relies on. If you find yourself reaching for `clone` here, that's the tell you've mismatched the pattern to the requirement.

Scenario 5: "Design an HTTP client builder that supports optionally setting headers, timeout, retries, and auth, in any combination, fluently." → **Builder**, not Prototype. Many optional, combinable construction parameters with no existing instance to copy from is Builder's signature, not Prototype's.

✓ **Before you move on:** Without looking up, state in one sentence each why scenario 2 isn't Prototype and why scenario 4 isn't Prototype — these are the two mismatches interviewers most often probe for.

▢ DEEP DIVE — read once, then use as reference

Path map: Fundamentals → Engineering Problem → Internals → Design (UML/Components) → Implementation (PHP/Laravel) → Production (scenarios + ADR) → Field Notes → Analogies/Architecture → SOLID/Performance/Concurrency → Trade-offs → Comparisons → Bugs/AI-review/Testing → Refactoring Journey → Practices/Mistakes/Traps → Interview Bank

Part 7 — Fundamentals

Definition: Prototype is a Creational design pattern (GoF, 1994) that creates new objects by cloning an existing, fully-configured instance — the *prototype* — instead of constructing a fresh object from raw inputs every time.

```
Traditional creation:  new ClassName(...args)    → constructor runs every time
Prototype creation:   clone $existingObject      → constructor runs ONCE, ever
```

The problem it solves: two related problems, one mechanism. First, expensive construction repeated many times — a constructor that hits a database, reads a config file, parses a template, or prepares a cryptographic context is fine to pay for once, not fine to pay for 50,000 times in a flash-sale batch. Second, needing "another object just like this one" without a clean Factory-style branching decision to encapsulate.

Beginner framing: think of a rubber stamp. Carving it takes effort; once it exists, stamping paper takes a second. You don't re-carve the stamp for every document — Prototype is "carve once, stamp many," applied to objects instead of ink.

Senior/staff framing: Prototype decouples the *cost of object construction* from the *number of objects you need* — converting an $O(n)$ initialization cost into $O(1)$ initialization plus $O(n)$ cheap copies. The trade-off you're buying is the responsibility of correctly defining what "copy" means for that object graph: shallow copy is a memory-level default that is frequently *wrong* for nested object references, and getting that wrong is the single most common production bug tied to this pattern. Leading with that trade-off — not just the benefit — is what separates a senior answer from a mid-level one.

✓ **Before you move on:** (1) State the GoF category and one other pattern in the same category. (2) In one sentence, what's the "senior" framing that a "mid" answer usually misses?

Part 8 — The Engineering Problem & Refactoring Trigger

What code looks like before this pattern: a constructor that mixes two categories of data that change at different rates — rarely-changing configuration (branding, tax rules, templates) and frequently-changing instance data (order ID, customer, amount) — and re-fetches/re-parses the rarely-changing part on every single call.

```
foreach ($orders as $order) {
    $invoice = new Invoice(
        companyName: fetchCompanyBranding(),           // network call, same every time
        taxProfile: fetchTaxProfile($order->region),    // network call, same every time
        pdfTemplate: loadPdfTemplate(),                 // disk/S3 read, same every time
        orderId: $order->id,                             // actually varies
        customerName: $order->customerName,             // actually varies
        amount: $order->amount,                         // actually varies
    );
}
```

Production-mindset questions — this is what actually separates a senior candidate's

answer here: - *What production problem actually forces this?* Not "slow constructors" in the abstract — a specific, measurable symptom: p99 latency on invoice generation climbing specifically during high-order-volume windows, traced via profiling to redundant calls to the branding/tax/template services rather than to the domain logic itself. - *How would a senior engineer discover this before it's a crisis?* By noticing the shape of the cost in a profiler or APM trace — the same three calls repeating identically across thousands of consecutive invocations — rather than waiting for a flash-sale incident to surface it. - *What metric would show it coming?* A rising ratio of "time spent in construction" to "time spent in actual domain logic" as order volume scales, or a flat per-order latency that should be dropping with caching but isn't. - *What would a competent engineer try first, and why might they reject it?* Caching the individual expensive fields (branding, tax rate) behind a generic cache layer — this works, but conflates caching concerns with the `Invoice` class's actual responsibility, and doesn't generalize as more expensive fields are added. This is a reasonable intermediate step, not a wrong one — but it's not the structural fix.

The refactoring trigger: the insight that leads to Prototype is noticing the constructor is doing two jobs — assembling expensive shared state, and recording cheap instance-specific state — and splitting them along the `clone` boundary: build the expensive object once, then `clone` it per instance, overwriting only what's actually instance-specific. (Full code sequence for this journey: Part 19.)

✓ **Before you move on:** (1) What's the specific profiler signature that should make you suspect this problem, as opposed to a general "slow constructor" complaint? (2) Why is the caching-individual-fields approach a reasonable first step but not the structural fix?

Part 9 — Internal Working

This pattern *does* have a genuine internals story in PHP, so it's worth the space — but trimmed to exactly what explains the core gotcha, not a general PHP-internals essay.

PHP objects are heap-allocated and accessed through reference-counted handles — a PHP object variable is never the object's data itself, it's a handle pointing at heap-allocated data. `clone` allocates a *new* object on the heap and bitwise-copies the original's property values into it: scalar properties (int, string, bool, float) are copied by value automatically and are always safe; object-typed properties are copied as **shared handles** — the clone's property points at the *exact same* heap object as the original's property, unless `__clone()` says otherwise. Copy-on-write applies to PHP arrays and strings, not to object graphs — assigning an array is cheap

until mutated, but copying an object handle just gives you two handles to one thing, with no COW protection.

```
BEFORE clone: $original → [Customer #42] → address → [Address #7]

AFTER shallow clone (no __clone() override):
$original → [Customer #42] └─┬─> BOTH point to [Address #7]
$copy      → [Customer #99] └─┘

Mutating $copy->address->city also changes what $original->address->city
reads – there is only ONE Address object, and both handles reference it.
```

`__clone()` runs **after** the shallow copy already exists, on the new object — it's a repair hook, not an interceptor. Its entire job is finding every object-typed (or array-of-objects) property and explicitly re-cloning it, cascading into nested objects' own `__clone()` if the graph is more than one level deep.

✓ **Before you move on:** (1) What PHP mechanism protects arrays from the shallow-copy problem, and why doesn't it protect object properties the same way? (2) When exactly does `__clone()` execute relative to the copy?

Part 10 — Components, UML & Language Mapping

Component	Responsibility
Prototype (interface, optional in PHP)	Declares the cloning contract — in PHP <code>clone</code> is a language keyword available on every object, so a formal interface is optional but useful once multiple prototype classes exist in a registry
ConcretePrototype	Holds expensive-to-build shared state; implements <code>__clone()</code> to deep-copy mutable nested objects
Client	Requests a clone, supplies instance-specific overrides
PrototypeRegistry (common in production, not in the original GoF diagram)	Maps a key (tenant/region/template variant) to the correct pre-built prototype

Class diagram:

```

«interface» Prototype
+-----+
| + clone(): static |
+-----+

    Δ

+-----+
|   InvoicePrototype   |
+-----+
| - taxProfile: TaxProfile | ← mutable, needs __clone() re-clone
| - pdfTemplate: string    | ← scalar, safe by default
| - orderId: int           | ← instance-specific
+-----+
| + __clone(): void        |
| + withOrder(...): static |
+-----+

```

A sequence diagram adds no real information beyond "clone, then overwrite fields" for this pattern — skipping it rather than including one for coverage's sake.

Language mapping — the core mechanism, portable:

Language	Mechanism	Deep-copy responsibility
PHP	<code>clone</code> + <code>__clone()</code>	Manually re-clone object properties inside <code>__clone()</code>
Java	<code>Cloneable</code> + <code>clone()</code> override, or a copy constructor	<code>Object.clone()</code> is also shallow by default; deep copy needs manual field-by-field cloning, same gotcha as PHP
Python	<code>copy.copy()</code> (shallow) vs. <code>copy.deepcopy()</code> (deep)	Python makes the shallow/deep distinction explicit in the API itself — arguably clearer than PHP's single <code>clone</code> keyword
Go	No built-in clone; manual struct copy (<code>newStruct := original</code>) copies value types only	Pointer-typed fields are shared references after a naive struct copy — identical gotcha, different syntax
TypeScript/ Node	<code>Spread ({...obj})</code> or <code>Object.assign()</code> for shallow; <code>structuredClone()</code> for deep	<code>structuredClone()</code> clones data but not class methods/prototypes — matters if the "prototype" object has behavior, not just state

The gotcha is universal — every mainstream language's default/naive copy mechanism is shallow. This is worth saying explicitly if the interview isn't in PHP: the reasoning transfers completely, only the syntax changes.

✓ **Before you move on:** (1) Name the PHP mechanism and its closest Java equivalent. (2) Which language makes the shallow/deep distinction explicit in its API naming, rather than requiring a magic method?

Part 11 — Implementation Overview (PHP/Laravel/Node)

All runnable code lives in `Prototype.php`. It progresses through three tiers: basic clone mechanics (isolating the language mechanic — proving `clone` skips `__construct()`), the shallow-copy bug shown failing then fixed (`Customer/Address`), and a production-shaped invoice-registry example. Walk through the file directly for the code; this section covers the framework reality.

Does Laravel use Prototype internally? — verified, not recalled. The commonly repeated claim is "Eloquent's `replicate()` is Prototype via `clone`." Having checked Laravel's actual source (`Illuminate\Database\Eloquent\Model`, current `master`), **this is only half right, and the half that's wrong matters:** `replicate()` does **not** use PHP's `clone`/`__clone()` mechanism at all. It builds a new instance via `new static`, computes the attribute set to carry over by explicitly excluding the primary key, timestamp columns, and any unique-ID columns, sets those filtered attributes directly with `setRawAttributes()`, copies the loaded `$relations` array onto the new instance, and fires a `replicating` model event:

```
public function replicate(?array $except = null)
{
    $defaults = array_values(array_filter([
        $this->getKeyName(), $this->getCreatedAtColumn(),
        $this->getUpdatedAtColumn(), ...$this->uniqueIds(), 'laravel_through_key',
    ]));
    $attributes = Arr::except($this->getAttributes(), $except ?
array_unique(array_merge($except, $defaults)) : $defaults);

    return tap(new static, function ($instance) use ($attributes) {
        $instance->setRawAttributes($attributes);
        $instance->setRelations($this->relations);
        $instance->fireModelEvent('replicating', false);
    });
}
```

Two things worth stating precisely in an interview rather than repeating the common "Laravel clones models" simplification: first, this is Prototype *in intent* (avoid rebuilding a model from raw inputs, seed a new instance from existing state, skip re-running whatever validation/hydration logic a fresh `new` + manual population would imply) but not Prototype *in mechanism* — no `clone` keyword involved. Second, `setRelations($this->relations)` copies the relations array by value, but any *related model objects already loaded inside it* are shared object references between the original and the replica — the exact shallow-copy gotcha from Part 9, present in real Laravel source, just implemented via manual assignment instead of `clone`. If you replicate a model with loaded relations and then mutate a related model on the replica, you can affect the original's loaded relation too, until saved.

Where Laravel genuinely doesn't use this pattern: the Service Container (Factory/DI-shaped) and Eloquent's query builder (Builder-shaped) dominate Laravel's actual creational-pattern usage — don't force a Prototype connection beyond `replicate()`.

Node.js: no built-in `clone`; `{...obj}`/`Object.assign()` for shallow, `structuredClone()` (Node 17+) for deep — but `structuredClone()` clones data only, not class instances with methods, which matters if your "prototype" object carries behavior.

✓ **Before you move on:** (1) Does Laravel's `replicate()` use PHP's `clone` keyword? (2) What's the shallow-copy risk specific to `replicate()`'s handling of loaded relations?

Part 12 — Where This Shows Up in Production

Scenario: flash-sale invoice generation. Branding, tax rules, and PDF templates are expensive and identical per (country, currency) pair; order data is cheap and always different. One `InvoicePrototype` per jurisdiction, registry-keyed, cloned per order. Trade-off: registry staleness becomes a new, monitored concern that didn't exist before — if tax rules change intraday, stale prototypes silently produce stale invoices until the registry entry is rebuilt.

Scenario: notification template fan-out. A billing event fans out to email/SMS/push, each needing a channel-specific but structurally similar object pre-loaded with branding, localization strings, and delivery config. Same cost shape as invoicing — mostly-static setup, small varying payload.

Service	Plausible fit	Why
Invoice/Billing	Strong	Expensive shared config, high per-request volume
Notification	Strong	Same cost shape
Order (reorder feature)	Moderate	Cloning a draft-order template maps directly to Laravel's <code>replicate()</code>
Payment	Anti-pattern	Cloning transaction state risks carrying stale/incorrect state into a financial record — construct freshly, always
Auth/Session	Anti-pattern	Cloning session/token state risks leaking shared auth state across requests

ADR — Architecture Decision Record (worked example):

Title: Use a Prototype Registry for multi-region invoice generation **Context:** Invoice generation latency dominates order-processing p99 during high-volume windows; profiling shows redundant branding/tax/template fetches, not domain-logic cost, as the driver. **Decision:** Introduce a `PrototypeRegistry` keyed by (country, currency), each entry a fully-configured `InvoicePrototype`. Orders clone the matching entry and overwrite only order-specific fields. **Alternatives considered:** (1) Cache individual fields (branding, tax rate) behind a generic cache — rejected as treating the symptom, not the structural cause, and doesn't reduce per-order object-assembly cost. (2) Precompute and cache fully-rendered invoices — rejected because every invoice's order-specific data is genuinely unique; there's no valid cache key across orders. **Consequences:** Construction cost drops from $O(n)$ to $O(1)$ + cheap clones. New operational responsibility: registry staleness must be actively invalidated (event-driven, on a `TaxRulesUpdated` domain event) rather than assumed correct forever. **Trade-offs:** Correctness now depends on `__clone()` being exhaustively correct for every mutable nested property — a missed one is a silent data-leak bug, not a crash, which is a materially worse failure mode to have introduced.

✓ **Before you move on:** (1) Name one service where Prototype is a genuine anti-pattern, and why. (2) In the ADR, what alternative was rejected, and on what basis?

Part 13 — Field Notes (Simulated Production Experience)

This is a rehearsal scaffold, not a script. Personalize it with details from your own projects before using it as an interview answer, or present it plainly as illustrative reasoning rather than claim it as your personal history — a follow-up question will expose a memorized-but-not-lived story fast.

"On a multi-region invoicing service, invoice generation was originally a per-order constructor call pulling branding, tax rules, and a PDF template inline. It worked at normal volume. During a regional flash-sale event, order volume spiked for a sustained window, and invoice generation became the dominant contributor to p99 order-processing latency. Profiling showed the same branding/tax/template fetches repeating identically across thousands of consecutive orders from the same region — not domain-logic cost.

The fix was a `PrototypeRegistry` keyed by (country, currency): one fully-configured prototype per key, cloned per order, with only order-specific fields overwritten. The registry subscribed to a `TaxRulesUpdated` domain event to invalidate and rebuild the relevant entry, rather than relying on a blind TTL.

The part that actually required care wasn't the cloning mechanism — it was correctly identifying every mutable nested object (a nested `TaxProfile`, specifically) that needed an explicit deep-copy inside `__clone()`. We initially missed one, and it surfaced as an intermittent, hard-to-reproduce bug where one customer's invoice adjustment appeared to leak into another's under concurrent load. What I'd do differently: write the clone-independence test (Part 18) *before* shipping the registry, not after the incident."

✓ **Before you move on:** (1) What specifically caused the production incident in this account — the pattern choice, or the implementation? (2) What single artifact would have caught it before shipping?

Part 14 — Analogies & Architecture Fit

Analogies: - **Passport office** — master template (logo, watermark, QR layout) stays constant; each passport is a "clone" with name/DOB/number changed. The office never redraws the watermark from scratch. - **Rubber stamp** — carving = expensive one-time construction; stamping = cheap repeated cloning. The clearest single intuition for "pay once, reuse many." - **Photocopier with a master document** — you copy the master and annotate the copy, rather than retyping the whole document each time. - **Cell division** — a cell copies existing DNA rather than building it from raw materials — useful for explaining that cloning-based creation is different *in kind* from constructing from scratch, not just faster.

Architecture fit: - **Clean/Hexagonal/Onion:** belongs in the domain/application layer — cloning is a domain-level decision about object identity and cost, not an infrastructure concern. A registry that *builds* prototypes may depend on infrastructure during construction; the clone

operation itself should stay infrastructure-free. - **DDD**: maps cleanly onto Value Objects that are expensive to construct — an immutable Value Object is trivially safe to share or clone, since there's no mutable state to leak (ties directly to Part 16's structural fix). - **Event-driven architecture**: a registry's invalidation trigger is a natural event-consumer role (Part 12's ADR) — keeps "when do I go stale" explicit rather than buried in a TTL with no domain meaning. - **CQRS**: no meaningful connection — stated plainly rather than forced. - **Cloud-native/Kubernetes**: the *principle* (baseline template + cheap parameterized duplication) underlies manifest templating broadly (Helm charts, Terraform modules), but this is a principle-level analogy, not literal object cloning inside a running service — worth one sentence if a system-design-adjacent interview goes there, not more.

✓ **Before you move on**: (1) Which analogy best captures "pay once, reuse many," and why? (2) Which architecture style has no meaningful connection to this pattern, and how should you say that in an interview?

Part 15 — SOLID, Performance & Concurrency

SOLID: SRP and OCP get a strong, positive connection — construction logic separates cleanly from "produce a new instance," and new variants (a new country) are added via a new registry entry, not by modifying existing classes. LSP requires discipline if `ConcretePrototype` subclasses exist — a subclass's `__clone()` override must not skip re-cloning what the parent class also needs cloned. ISP and DIP have no meaningful connection worth forcing here — PHP's `clone` doesn't create a fat-interface problem, and there's no significant dependency-direction question this pattern resolves on its own.

Performance: without Prototype, N instances cost $O(N \times \text{construction_cost})$; with it, $O(1 \times \text{construction_cost}) + O(N \times \text{clone_cost})$. The win is real specifically because `clone_cost` (a memory-level copy) is typically far cheaper than `construction_cost` when construction involves I/O or parsing — but this is a reasoned estimate based on the relative cost of memory operations vs. I/O, not a benchmark; validate with real profiling before quoting a specific number in a design review. Memory trade-off: every clone is an independent heap allocation, which can shift the bottleneck to memory pressure at very high clone volume with very large object graphs — worth monitoring, not assuming away.

Concurrency: under standard PHP-FPM (per-request, no shared memory between requests), a registry read is safe without additional locking. This changes under long-running processes — Swoole, RoadRunner, persistent queue workers — where a registry instance persists across many requests in the same process. If `__clone()` is incomplete, two logically-independent operations executing in that shared process can end up mutating the same underlying nested object — a genuine race condition, not just a bug, specifically in that deployment shape. In Node.js, the single-threaded event loop doesn't remove the equivalent risk — interleaved async operations can still observe a shared, incompletely-cloned reference; "single-threaded" isn't the same guarantee as "no shared-mutable-state bugs."

✓ **Before you move on**: (1) Which two SOLID principles have no meaningful connection to this pattern? (2) Under which specific PHP deployment models does an incomplete `__clone()` become a concurrency bug, not just a correctness bug?

Part 16 — Advantages, Disadvantages & Trade-offs

Dimension	Advantage	Disadvantage / trade-off
Performance	Converts $O(N)$ construction into $O(1)$ + cheap copies for expensive-construction cases	Pure overhead for cheap-to-construct objects; needs profiling data to justify, not assumption
Scalability	Improves throughput under load for creation-heavy hot paths	A shared registry in a long-running process can itself become a bottleneck or race-condition source if <code>__clone()</code> is incomplete
Maintainability	Centralizes construction logic in one place	Every new mutable nested property is a new place a deep-copy can be silently forgotten — an asymmetry that compounds over time
Readability	Well-understood, named pattern once recognized	Slightly less direct than a plain <code>new</code> for engineers unfamiliar with clone semantics
Security	Neutral-to-positive for config/template-shaped data	Actively risky if misapplied to session/auth/transaction-shaped data (Part 3)
Testing	Construction logic is testable once, in isolation	Adds a real, non-optional new testing obligation — clone-independence tests (Part 18) — that a naive <code>new</code> -based design never required
Observability	A registry is a natural place to add metrics (entry age, clone count per key)	Registry staleness is a new operational failure mode that didn't exist before the pattern was introduced

✓ **Before you move on:** (1) Name one dimension where this pattern is a clear net positive with no real downside. (2) Name one dimension where the trade-off genuinely could go either way depending on context.

Part 17 — Pattern Comparisons

	Prototype	Factory	Builder	Singleton
Mechanism	<code>clone</code> an existing instance	<code>new</code> via creation logic/branching	Step-by-step assembly, fluent API	Exactly one instance, ever
Best for	Expensive, structurally-similar instances	Deciding <i>which</i> concrete type to build	Objects with many optional/combinable parameters	A single shared instance globally
Relationship to <code>clone</code>	Central mechanism	Unrelated	Unrelated	Actively blocks it — <code>__clone()</code> is typically made <code>private</code> specifically to prevent bypassing the single-instance guarantee
	—			No — opposite instinct

	Prototype	Factory	Builder	Singleton
Cooperates with Prototype?		Often — a Factory frequently builds the initial prototype that's then cloned	Rarely relevant together	

Memento, briefly: commonly confused because both "involve copying state," but the intent differs — Memento captures one object's internal state *for later restoration of that same object* (undo/redo); Prototype captures state *to seed brand-new, independent objects going forward*. Memento is time-travel for one object's history; Prototype is mass-production from a baseline.

Decision table:

Situation	Reach for
Construction is expensive; instances are near-identical	Prototype
Concrete type decided at runtime via branching logic	Factory
Many optional, combinable construction parameters	Builder
Exactly one instance must ever exist	Singleton
Snapshot and later restore one specific object's past state	Memento
A Factory needs to build the initial object that Prototype then clones	Both, together

✓ **Before you move on:** (1) Why does Singleton typically make `__clone()` private? (2) What's the one-sentence distinction between Prototype and Memento?

Part 18 — Production Bugs, AI-Generated Code Review & Testing

The flagship bug — shallow-copy leak. A mutable nested object property wasn't deep-copied in `__clone()`. Symptom: mutating what looks like an independent clone silently changes data on a different object — often surfacing as "customer A's data appeared on customer B's invoice" under concurrent/high-volume conditions. Debug by reproducing with an identity check (`===`, not `==`) between the clone's nested property and the original's — this is the same technique as the preventive test below, just applied reactively.

Stale registry entries. New orders keep using outdated tax rates or branding after a legitimate business change, with no crash — a silent correctness bug that may only surface in an audit. Fix: event-driven invalidation (Part 12's ADR), not a shorter TTL, which just trades staleness for more redundant rebuilding.

How AI coding assistants typically get this pattern wrong — worth its own callout, since reviewing AI-generated code is now a standard part of the job at Staff+ level: - **Most common failure:** AI-generated `clone / __clone()` implementations frequently handle the *first* level of an object graph correctly but don't cascade into a second level — e.g., correctly cloning a `Customer`'s `Address` but missing that `Address` itself holds a mutable `GeoCoordinates` object one level deeper. This mirrors exactly the "looks finished but isn't" failure mode a human engineer falls into (Part 19's Stage 3), and AI tools reproduce it reliably because the surface pattern

("override `__clone()`, clone the object property") is easy to generate without reasoning about graph depth. - **Second most common failure:** AI-suggested "Prototype" implementations for scenarios that are actually Factory problems — e.g., generating a `clone`-based solution for "create the right payment method object based on type," which should branch on type via Factory, not duplicate a template via clone. This happens because both involve "creating an object," and the model pattern-matches on surface similarity rather than on whether the concrete type genuinely varies. - **What a reviewer should check before merging:** (1) does `__clone()` cascade to every level of the object graph, verified with an identity-check test, not just present at the top level; (2) is the chosen pattern actually justified by "expensive construction + similar instances," or was `clone` reached for because the AI was asked to "create objects efficiently" without that specific justification; (3) does the generated code clone anything from the anti-trigger list (Part 3) — AI assistants have no inherent judgment about session/auth/transaction-state risk unless explicitly prompted for it.

Testing strategy — the clone-independence test is the one category that matters most for this pattern:

```
public function test_cloning_produces_a_fully_independent_object(): void
{
    $prototype = InvoicePrototype::fromConfig($fakeConfig);
    $cloneA = clone $prototype;
    $cloneB = clone $prototype;

    $cloneA->taxProfile->rate = 0.99;

    $this->assertNotSame($cloneA->taxProfile, $cloneB->taxProfile); // identity, not
value
    $this->assertNotSame($cloneA->taxProfile, $prototype->taxProfile);
    $this->assertNotEquals(0.99, $cloneB->taxProfile->rate);
}
```

The critical detail: `assertNotSame` (identity) on every nested object property, not `assertNotEquals` on values alone — a value check can pass right up until the moment one clone is mutated. Run one such test per mutable nested property; a new mutable property added without a matching test is a code-review blocker.

Code review checklist: every object-typed or array-of-objects property has a corresponding `__clone()` line, or an explicit comment stating it's deliberately shared because it's immutable; a clone-independence test exists for every mutable nested property touched in the PR; nothing from the anti-trigger list (Part 3) is being cloned; any registry entry sourced from mutable external data has a defined invalidation path, not just "it'll restart eventually."

✓ **Before you move on:** (1) What's the single most common way AI tools get this pattern wrong? (2) Why must the clone-independence test use `assertNotSame`, not `assertEquals`?

Part 19 — Refactoring Journey

Full code for every stage lives in `Prototype.php`; this narrates the reasoning connecting each one.

Stage 1 — Terrible (*where most engineers start, no shame in it*): everything rebuilt inline, every call, no separation of concerns. Works, doesn't scale.

Stage 2 — Bad, but a realistic first instinct (*often written by a mid-level engineer under time pressure*): hand-rolled static caching of individual constructor arguments. Papers over the symptom without addressing the structural cause, and doesn't generalize as more expensive fields appear.

Stage 3 — Average, and the most dangerous stage in the whole journey (*a senior engineer moving fast, or code that later drifts as properties are added without matching review*): correctly splits expensive-shared from cheap-instance-specific construction, but implements only a shallow clone with no `__clone()` override. Passes casual testing, looks finished, silently carries the shallow-copy bug the moment any nested object exists.

Stage 4 — Pattern correctly applied (*what a rigorous senior/staff engineer ships*): adds a correctly-cascading `__clone()` plus a clone-independence test proving it. Functionally complete and correct.

Stage 5 — Production-ready (*staff-level judgment about the surrounding system, not just the class*): wraps the correct implementation in a keyed registry with event-driven invalidation, instrumented with entry-age and clone-count metrics, covered by the full test pyramid including a multi-tenant variant if applicable. This is the version that survives a flash sale and an on-call rotation, not just a code review.

✓ **Before you move on:** (1) Which stage is the most dangerous to leave in production, and why specifically that one rather than Stage 1 or 2? (2) What distinguishes Stage 4 from Stage 5 — is it a code difference or a systems difference?

Part 20 — Practices, Mistakes & Traps

Junior mistakes: believing `clone` re-runs `__construct()`; reaching for Prototype on trivially cheap objects "because it's a pattern I know"; not knowing `__clone()` exists and manually copying fields wherever a copy is needed instead.

Mid-level mistakes: implementing `__clone()` for the top level but forgetting it must cascade into nested objects' own `__clone()` for graphs deeper than one level; assuming "I overrode `__clone()`" automatically means "it's fully deep now" without verifying every mutable property was addressed; building a registry with no invalidation strategy at all.

Senior mistakes — subtler, and the ones that actually distinguish levels in an interview: applying Prototype to session/auth/transaction-shaped objects because the *mechanism* technically fits, without recognizing the security anti-pattern from first principles rather than from having been burned by it once; missing the concurrency escalation specific to long-running processes (correctly reasoning about PHP-FPM, but not adjusting that reasoning for Swoole/RoadRunner); stating a performance claim as benchmarked fact in a design review without having actually measured it for the system in question.

Interview traps — the specific follow-ups that catch memorized-but-shallow understanding: - "So cloning is always faster than `new`, right?" — agreeing unconditionally is the trap; name the condition (expensive construction) under which it's true. - "Fix this shallow-copy bug" (live) — the trap is fixing only the first-level nested property and declaring victory

when the graph is actually two or three levels deep. - "Is PHP thread-safe here?" — a blanket "PHP is single-threaded, no concurrency issue" answer is the trap; the correct answer distinguishes PHP-FPM from Swoole/RoadRunner/persistent workers. - "Would you clone this session object?" — testing judgment, not mechanism; agreeing without pushback is the red flag interviewers are listening for. - "So Prototype replaces Factory?" — framing them as competitors instead of correctly noting they solve different problems and often cooperate.

✓ **Before you move on:** (1) Which junior mistake is purely a language-mechanics gap, not a design judgment gap? (2) Pick one interview trap above and state the correct answer in one sentence, from memory.

Part 21 — Interview Question Bank & Coding Problems

22 questions total across five levels — intentionally below a flat quota, because this pattern's real interview footprint (Part 2) doesn't justify padding it to match a Very High priority pattern's bank.

Beginner (5)

B1. What is the Prototype pattern? *Wrong:* "Copying an object." *Good:* "A creational pattern that clones an existing instance instead of building from scratch." *Excellent:* Good + "used specifically when construction is expensive or many similar instances are needed quickly." *Follow-up:* Name the other four creational GoF patterns.

B2. Does `clone` call the constructor in PHP? *Wrong:* Yes. *Good:* "No — it bitwise-copies properties and calls `__clone()` if defined." *Excellent:* Good + a concrete example of why that matters. *Follow-up:* What does `__clone()` actually do, and when does it run?

B3. Is PHP's `clone` shallow or deep by default? *Wrong:* Deep. *Good:* "Shallow — scalars by value, objects by shared reference." *Excellent:* Good + a concrete bug example and the fix. *Follow-up:* Show the fix live.

B4. When would you NOT use Prototype? *Wrong:* No real answer. *Good:* "When the object is cheap to construct." *Excellent:* Good + the session/transaction-state anti-trigger. *Follow-up:* What would you use instead if concrete types genuinely vary?

B5. Name one production use case. *Wrong:* A Shape/Animal toy example. *Good:* Invoice generation with shared branding/tax config. *Excellent:* Good + explains specifically what's expensive about the shared part. *Follow-up:* What goes wrong if the cloning is implemented incorrectly there?

Intermediate (5)

I1. Walk me through fixing a shallow-copy bug. *Wrong:* Vague "just override clone." *Good:* Identify the shared nested object, override `__clone()`, explicitly re-clone it. *Excellent:* Good + writes an identity-check test proving the fix. *Follow-up:* What if that nested object has its own nested objects?

I2. How does Prototype relate to Factory? *Wrong:* "They're alternatives, pick one." *Good:* Correctly states they solve different problems. *Excellent:* Good + names that a Factory often builds the initial prototype that Prototype then clones. *Follow-up:* Design a system using both together.

13. How would you support multiple prototype variants (e.g., per-country configs)?

Wrong: "Add if/else branches on country." *Good:* A keyed registry. *Excellent:* Good + discusses invalidation when underlying config changes. *Follow-up:* What happens if config changes while the registry is live?

14. Is cloning thread-safe? *Wrong:* Blanket yes/no. *Good:* Fine under PHP-FPM; needs care in long-running processes. *Excellent:* Good + explains *why* — a shared registry in a persistent process turns an incomplete `__clone()` into a race condition. *Follow-up:* How does this differ in Node.js?

15. Does Laravel use this pattern? *Wrong:* Confidently claims Eloquent "clones" models. *Good:* Names `replicate()` as the analogue. *Excellent:* Good + correctly states `replicate()` uses `new static()` + manual attribute filtering, not `clone / __clone()`, and flags the shared-relations gotcha. *Follow-up:* What does `replicate()` exclude by default, and why?

Senior (5)

S1. Design a Prototype-based system for a specific expensive-construction scenario, and walk through your reasoning. *Wrong:* Jumps to code before identifying what's actually expensive. *Good:* Identifies the split, sketches a registry, mentions deep-copy correctness. *Excellent:* Good + proactively raises invalidation, testing, and an anti-trigger case unprompted. *Follow-up:* What breaks first at 10x the load you designed for?

S2. A customer-reported data leak turns out to be a shallow-copy bug already in production. Walk through your response. *Wrong:* Jumps straight to "fix `__clone()`" with no containment/blast-radius step. *Good:* Reproduce with an identity check, assess blast radius, fix, add a regression test. *Excellent:* Good + audits every other prototype class in the codebase for the same class of bug, not just the reported one. *Follow-up:* How would this differ in a multi-tenant system?

S3. When would you explicitly choose not to use Prototype even though it technically fits? *Wrong:* Can't name a real counter-case. *Good:* The security/transaction-state anti-trigger. *Excellent:* Good + the deeply-nested-graph case, proposing immutability as the structural alternative. *Follow-up:* What would you tell a teammate who insists on using it for a session object anyway?

S4. How would you make deep-copy correctness structurally enforced rather than relying on developer discipline? *Wrong:* "Code review should catch it." *Good:* Mandatory clone-independence tests per mutable property. *Excellent:* Good + proposes immutability by default as the fix that removes the need for discipline entirely. *Follow-up:* Could static analysis catch a missing deep-copy?

S5. A junior engineer proposes cloning a `PaymentTransaction` to quickly retry a failed payment. Response? *Wrong:* Approves it as "valid Prototype usage." *Good:* Flags the risk of carrying over stale transaction state. *Excellent:* Good + proposes constructing a fresh transaction that references the failed one's ID for audit purposes instead. *Follow-up:* What regulatory concerns apply in a fintech context?

Staff (4)

ST1. How would you decide whether to standardize Prototype usage across multiple teams? *Wrong:* "Tell every team to use it if construction is slow." *Good:* A shared base class/trait encoding correct `__clone()` discipline and the test pattern. *Excellent:* Good + weighs the

coupling cost of a shared abstraction against the correctness benefit, proposing opt-in with clear guidance rather than a mandate. *Follow-up*: How would you roll it out without a big-bang migration?

ST2. How would you detect latent shallow-copy bugs across a large microservices fleet without manually auditing every codebase? *Wrong*: "Have every team review their own code." *Good*: A linter rule flagging object-typed properties with no `__clone()` override. *Excellent*: Good + acknowledges static analysis can flag *missing* overrides but not verify a *present* one is fully correct, and pairs it with a shared test-helper trait. *Follow-up*: CI-blocking or advisory?

ST3. Would you build a general-purpose `PrototypeRegistry` as shared platform infrastructure, or let each service own its own? *Wrong*: Picks a side with no engagement with the trade-off. *Good*: Names the standardization-vs-coupling trade-off. *Excellent*: Good + proposes a concrete decision rule (e.g., shared if 3+ teams have the same shape). *Follow-up*: Who owns the on-call burden for a centrally-built registry?

ST4. How does a region-keyed prototype registry interact with data-residency requirements in a regulated market? *Wrong*: No connection made. *Good*: Notes region-keying naturally aligns with residency boundaries if construction dependencies stay in-region. *Excellent*: Good + flags the risk of a centrally-built registry inadvertently routing construction through infrastructure in the wrong jurisdiction. *Follow-up*: How would you audit this after the fact?

Principal (3)

P1. Describe a case where a correctly-chosen pattern still caused a production incident. What does that teach about pattern usage broadly? *Wrong*: Claims correctly-applied patterns are inherently safe. *Good*: Narrates an incident where the pattern was right but an implementation detail (a missed nested property) still caused it. *Excellent*: Good + generalizes: a pattern name describes intent, not a correctness guarantee — that has to come from tests and review. *Follow-up*: How do you instill that in engineers eager to apply patterns?

P2. Design review: a team wants to clone a `User` aggregate, including a mutable nested `Permissions` object, for a "duplicate user for testing" admin feature. Your feedback? *Wrong*: Approves without probing `Permissions`'s mutability. *Good*: Requires an explicit deep-copy and independence test before approval. *Excellent*: Good + questions whether cloning is the right feature shape at all, proposing a "create from template with freshly-assigned permissions" flow instead — reducing risk at the design level, not just the implementation level. *Follow-up*: How do you phase this feedback so the team doesn't feel blocked?

P3. Summarize this entire pattern, for a new-hire principal engineer, in under two minutes. *Wrong*: Surface-level definition only. *Good*: Covers the mechanism, the shallow-copy risk, one production example. *Excellent*: Matches the 1-minute pitch in Part 4 almost exactly — the point being: if you can produce that unprompted here, you've internalized the handbook rather than memorized it. *Follow-up*: None — capstone question.

Coding problems (starter code + solutions in `Prototype.php`): (1) "Fix the Leak" — given a buggy `Customer / Address` pair, find and fix the shallow-copy bug, prove it with a test. (2) "Build a Registry" — given three country configs, implement a keyed registry that never leaks the

stored master. (3) "Deep Clone a Three-Level Graph" — `Order → Customer → Address` , each level individually responsible for cloning its own immediate object properties.

✓ **Before you move on:** (1) Which single question above is most likely to actually come up as a follow-up inside an unrelated LLD round, given Part 2's market data? (2) Can you deliver the Part 4 one-minute pitch from memory right now, unprompted?

▢ APPENDIX

Part 22 — Learning Roadmap & Self-Assessment

Roadmap: the PHP manual's `clone / __clone` pages (primary source for exact semantics); the original GoF book's Prototype chapter (for the canonical intent); Laravel's `Illuminate\Database\Eloquent\Model::replicate()` source directly (now verified in Part 11 — read it yourself to confirm and go further); re-implement this handbook's Tier 2 and Tier 3 examples from memory, then diff against `Prototype.php`.

MCQs

1. What does `clone` call by default? (A: *nothing beyond the bitwise copy, unless `__clone()` is defined*)
2. Which property type is safe to leave shallow-copied with zero risk? (A: *an immutable/readonly value object*)
3. What should `PrototypeRegistry::get()` return? (A: *a clone, never the stored master*)
4. Which PHP deployment model makes an incomplete `__clone()` a concurrency risk? (A: *long-running processes — Swoole/RoadRunner/persistent workers*)
5. Does Laravel's `replicate()` use `clone`? (A: *No — `new static()` + manual attribute filtering*)

Scenario questions: - A notification service clones a per-locale template containing a mutable `array $placeholders` holding objects. Under load, placeholder values leak across notifications. Diagnose and fix. → Even though it's an array (value-copied in PHP), objects *inside* it are still shared references after a shallow clone — deep-copy each object inside the array within `__clone()`, or restructure to hold only scalars if object identity was never needed. - A stakeholder asks why you can't just cache the fully-rendered invoice instead of using this pattern. Explain the distinction. → Full-result caching only works if the entire output is reusable as-is; here every invoice's order-specific data is genuinely unique, so only the *shared setup* — not the final output — is cacheable/cloneable.

Refactoring exercise: given a `ReportGenerator` whose constructor loads chart-styling config, a logo, and a data-source connection, then accepts report-specific parameters — refactor into a correctly-implemented Prototype following the Part 19 journey, writing the clone-independence test first. (Reference solution: `Prototype.php`, "Self-Assessment Exercise.")

Architecture/debugging scenario: a `PrototypeRegistry` deployed as a long-running RoadRunner process shared across regions (for cost efficiency) intermittently shows one region's currency symbol on another region's invoice under high concurrent load. Using Parts 9, 15, and 18: this is the shallow-copy leak escalated by the long-running-process concurrency framing — reproduce with a concurrent identity-check test simulating simultaneous cross-region requests, identify the incompletely-cloned property (likely a `CurrencyFormat` object), fix with a cascading `__clone()`, add a permanent concurrent-access regression test (not just single-threaded independence), and weigh whether region-isolated processes are the more durable long-term fix versus relying entirely on code-level correctness in a shared process.

This handbook intentionally ends here rather than padding further — per Part 2, Prototype is a Low-frequency pattern across your five target markets. Reinvest the prep time you saved here into Strategy, Factory, Observer, and Singleton.